Closing the Syntax Gap Between Python and Numba's nopython Mode

Rodrigo Martinez Pinto

***Abstract*—Numba is a widely adopted LLVM-based just-in-time compiler for Python that delivers near-C performance for numerical code. Its `nopython` mode, which produces fully compiled machine code without CPython involvement, is restricted to a subset of Python syntax that excludes f-strings, list comprehensions, keyword arguments, and the PEP 484 type-hint notation used by modern Python codebases. Developers who reach these boundaries receive opaque LLVM-level error messages requiring expert knowledge to interpret. This article describes the CloudSealed Compiler, an open-source layer that resolves these limitations through two AST-level source transformations applied before Numba compilation, a type mapper that automatically translates PEP 484 annotations into Numba IR types, and a structured diagnostic system producing actionable error messages. A dataclass-to-jitclass bridge and a per-function compilation profiler complete the toolchain. The system is available as an open-source package, tested with a 38-case test suite, and imposes fewer than 1.4 ms of one-time overhead per function at decoration time. The package is available at https://pypi.org/project/cloudsealed-jit.**

***Index Terms*—just-in-time compilation, LLVM, Python, abstract syntax tree, source transformation, type inference, Numba, nopython mode, PEP 484, ergonomics**

I. Introduction

Python has become the dominant language for scientific computing and data-intensive engineering, largely because its ecosystem—NumPy, SciPy, Pandas—offers high-level expressiveness with compiled-library performance for array operations. For code that cannot be expressed as array operations, Numba [1] provides a JIT compiler that translates decorated Python functions directly to LLVM IR and native machine code. In benchmarks representative of numerical kernels, Numba-compiled code runs within a factor of two of equivalent C code [2], [13].

Numba's performance comes with a strict constraint: the `nopython` mode that produces pure machine code supports only a restricted subset of Python. The following constructs, all legal and idiomatic in Python 3.10+, are rejected at compile time:

- **F-strings** (`f"value: {x:.2f}"`)—raises `TypingError` because Numba cannot lower `JoinedStr` AST nodes;
- **List comprehensions** (`[x*2 for x in arr]`)—raises `TypingError` in nopython mode;
- **Keyword arguments** at call sites (`f(a=1, b=2)`)—unsupported in compiled dispatch;
- **PEP 484 generic annotations** (`Optional[float]`, `List[int]`, `Dict[str,float]`)—not parsed as Numba IR types; must be manually written as string signatures [11].

These restrictions are not edge cases. A search of the Numba GitHub issue tracker returns over 200 open issues in which users report `TypingError` failures caused by f-strings, list comprehensions, or unsupported type annotations. The error messages produced in these situations are LLVM-level tracebacks of 30–60 lines. A representative message reads: *"Failed in nopython mode pipeline (step: nopython frontend), non-precise type pyobject."* This message does not identify the offending construct, name the function, or suggest a fix.

In practice, adopting Numba in a modern Python codebase means maintaining two versions of the same logic: one written in idiomatic Python 3, the other stripped of f-strings, list comprehensions, and type annotations so that nopython mode will accept it. Every update to the surrounding codebase risks breaking the Numba-compatible copy. This is not a theoretical concern; it is the reason many teams abandon Numba after the first refactor.

Contributions

This article makes the following contributions:

1) **FStringTransformer**: an `ast.NodeTransformer` subclass that rewrites `JoinedStr` nodes, including format-spec variants, into `str()` + concatenation chains that Numba accepts.
2) **ListCompTransformer**: an `ast.NodeTransformer` subclass that rewrites single-generator list comprehensions, including filter clauses, into explicit `for`-loop equivalents.
3) **`map_python_type_to_numba()`**: a type mapper that translates PEP 484 annotations—scalars, `np.ndarray`, `Optional[T]`, `List[T]`, `Tuple[T,...]`, `Dict[K,V]`—into Numba IR types, enabling ahead-of-time (AOT) compilation from standard type hints.
4) **CloudSealedCompileError**: a structured diagnostic wrapper that intercepts Numba compilation failures, identifies unsupported patterns by source inspection, and produces a human-readable error with a suggested fix.
5) **`@jitdataclass`**: a decorator that converts standard Python `@dataclass` definitions into Numba

R. Martinez Pinto is an independent software engineer and the founder of CloudSealed. E-mail: contact@cloudsealed.com. Source code: https://github.com/cloudsealed/JIT-Optimization-Engine. Package: `pip install cloudsealed-jit` (MIT license). The author has contributed patches to the Numba upstream compiler (pull requests #10880, #10881, and #10882).

`@jitclass` structs, preserving default values, user-defined methods, and nested dataclass field resolution.

6) **Compilation profiler**: a per-function registry exposing compile time, cache status, and Numba signature, accessible at runtime via `compilation_stats()`.

The complete implementation is available under the MIT license (`pip install cloudsealed-jit`). The test suite covers all six contributions across 38 test cases.

II. Background and Related Work

A. Numba and LLVM-Based Python JIT Compilation

Numba [1] works by tracing the types of a function's arguments at the first call and emitting LLVM IR specialized for those types. The `@njit` decorator (equivalent to `@jit(nopython=True)`) produces code entirely free of CPython objects, enabling SIMD vectorization and parallelization via OpenMP [12]. Numba has been widely applied in scientific computing [3] and high-performance data analysis pipelines.

The restriction to a Python subset is a fundamental consequence of Numba's type inference: for the compiler to emit machine code without CPython involvement, every value in the function must have a statically known LLVM type. Python constructs that introduce heterogeneous or dynamically typed objects—including f-strings, generators, and arbitrary class instances—cannot be lowered to LLVM IR.

B. Alternative Python-to-Native Compilers

Cython [4] compiles an annotated Python superset to C. It supports the full Python language but requires a build step, `.pyx` source files, and `.pxd` declaration files. Cython does not target LLVM and does not support SIMD or GPU dispatch natively. Seljebotn [5] demonstrated Cython's effectiveness for numerical kernels but noted its requirement for annotated source files as a usability barrier.

mypyc [6] compiles type-annotated Python to C extensions using mypy's type information. It supports f-strings and list comprehensions but requires a compilation step separate from the Python interpreter and does not produce LLVM IR.

PyPy [7] is an alternative Python interpreter with a tracing JIT [8], but it is not compatible with the CPython C extension ecosystem (NumPy, SciPy) and cannot be used as a drop-in module within a CPython process.

numba-dpex [9] extends Numba with data-parallel GPU extensions but inherits the same nopython-mode restrictions.

Zhang et al. [2] measured several JIT compilers for Python and found nopython mode delivers the highest speedup among current alternatives. That result makes the usability barrier more consequential, not less: the fastest tool is also the hardest to use.

None of the above address the specific problem of enabling idiomatic Python syntax in Numba's nopython mode without a separate build step or interpreter change. Table I summarizes the capabilities of each approach.

TABLE I
Feature comparison of Python-to-native compilation approaches. ✓ = supported, × = not supported, P = partial.

Feature	**@jit**	@njit	Cython	mypyc	PyPy
F-strings	✓	×	✓	✓	✓
List comprehensions	✓	×	✓	✓	✓
Keyword arguments	✓	×	✓	✓	✓
PEP 484 auto-signature	✓	×	P	✓	n/a
No build step required	✓	✓	×	×	n/a
LLVM / SIMD backend	✓	✓	×	×	✓
Pure-Python fallback	✓	×	×	×	n/a
NumPy ecosystem	✓	✓	✓	✓	×

C. AST Transformation in Python

Python's `ast` module [10] provides a standard interface for parsing, transforming, and unparsing Python source trees. The technique of rewriting a program before handing it to a downstream compiler is well established in compiled languages, but has seen little use in the Python JIT ecosystem. To our knowledge, no prior tool applies AST rewriting specifically to close the gap between idiomatic Python 3 and Numba's nopython mode.

III. Design

The CloudSealed Compiler is a preprocessing layer that intercepts a function before it reaches Numba. All transforms execute once at decoration time; there is no per-call overhead from the transformation pipeline.

A. Architecture

Figure 1 shows the ten-stage compilation pipeline applied by the `@jit` decorator. Stages 1–6 constitute the source transformation phase; stages 7–8 the Numba compilation phase; and stages 9–10 the runtime dispatch phase.

B. FStringTransformer

F-strings in Python compile to `JoinedStr` AST nodes containing a sequence of `Constant` and `FormattedValue` children. `FormattedValue` carries the expression, an optional conversion character, and an optional format spec.

The transformer visits each `JoinedStr` node and constructs a left-associative binary addition tree. Each `FormattedValue` is wrapped in a `Call` node invoking the built-in `str()` function. Format specs (e.g., `:.2f`) are discarded: Numba cannot lower float formatting, so the transform preserves type correctness—the value is converted to a string—at the cost of losing format precision, a documented trade-off.

Listing 1 shows the transform for a representative f-string.

```
# Input: Numba rejects this
msg = f"cost {amount:.2f} for {label}"

# Output: Numba accepts this
msg = "cost " + str(amount) + " for " + str(label)
```

Listing 1. FStringTransformer input and output.



Fig. 1. The CloudSealed Compiler pipeline. All stages execute once at decoration time except stage 9, which executes per call only when keyword arguments are present.

C. ListCompTransformer

The transformer operates on `Assign` statements whose value is a `ListComp` with exactly one generator. It rewrites the assignment into three statement nodes: (1) an assignment of an empty list literal to the target name; (2) an `If` chain for any filter clauses; and (3) a `For` loop whose body calls `target.append(element_expression)`.

Single-generator comprehensions cover the dominant case in numerical code. Multi-generator comprehensions are left untransformed; the semantics of nested scope would require a more involved rewrite, and such patterns are rare in the kernels where Numba is typically applied.

Listing 2 illustrates the rewrite.

```
# Input: Numba rejects this
result = [x * 2 for x in data if x > 0]

# Output: Numba accepts this
result = []
for x in data:
    if x > 0:
        result.append(x * 2)
```

Listing 2. ListCompTransformer input and output.

D. Type Mapper

`map_python_type_to_numba()` translates a Python type object into a Numba IR type. It handles scalar types (`int`, `float`, `bool`, `str`, NumPy scalar types); `np.ndarray`; and PEP 484 generic types (`Optional[T]`, `List[T]`, `Tuple[T,...]`, `Dict[K,V]`) via the `__origin__` and `__args__` introspection attributes [11]. When the mapper cannot resolve a type, it returns `None` and the decorator falls back to deferred Numba type inference.

The mapper is used by the `@jit` decorator to construct an AOT signature from `inspect.signature()`, enabling full ahead-of-time compilation with no manual string annotations.

E. @jitdataclass

Numba's `@jitclass` requires a manually written spec list of `(field, numba_type)` tuples and a handwritten `__init__`. The `@jitdataclass` decorator automates this bridge: it enumerates fields via `dataclasses.fields()`, maps each type through the type mapper (with a registry lookup for nested `@jitdataclass` fields), generates a clean `__init__` via `exec()` preserving default values, strips dunder methods that `@jitclass` would reject, and calls `jitclass(spec)(CleanClass)`. A module-level registry keyed by `__qualname__` enables fields of one `@jitdataclass` to reference another as a Numba `StructRef` type.

Listing 3 shows the two decorators used together, combining all four transforms in a single example.

```
from cloudsealed_jit import jit, jitdataclass

@jitdataclass
class Particle:
    x: float
    y: float
    mass: float = 1.0        # default value

@jit()
def kinetic_energy(p: Particle, v: float) -> float:
    # f-string: rewritten by FStringTransformer
    label = f"mass={p.mass}"
    # list comp: rewritten by ListCompTransformer
    components = [p.x ** 2, p.y ** 2]
    return 0.5 * p.mass * v ** 2
```

Listing 3. `@jitdataclass` and `@jit`: all transforms active.

F. Diagnostic Layer

`CloudSealedCompileError` wraps the `try/except` around the `njit()` call. On failure, it scans the function source text for six known unsupported patterns—set construction, `with` statements, `yield`, dictionary unpacking, inline class definitions, and `import` statements—and emits a human-readable message identifying the likely cause and suggesting the `NUMBA_DISABLE_JIT=1` environment variable as a debugging fallback.

IV. Evaluation

A. Experimental Setup

Measurements were taken on an x86-64 machine running Linux 5.15 and Python 3.11.9 with NumPy 1.26. Each benchmark was repeated 1,000 times; we report the median and 99th percentile. The AST transform and kwargs overhead benchmarks do not require Numba and are therefore independent of its availability; compilation-time measurements with Numba are not reported here, as they are highly dependent on LLVM version and hardware cache state.

B. Case Study: N-Body Force Kernel

To illustrate the practical effect of the transforms, consider a simplified n-body force kernel that computes pairwise gravitational forces between particles. A natural implementation in Python 3 uses a list comprehension to initialize the force array and an f-string to log per-particle status:

```
@jit()
def compute_forces(pos: np.ndarray,
                   mass: np.ndarray) -> np.ndarray:
    n = len(pos)
    forces = [0.0 for _ in range(n)]    # ListComp
    for i in range(n):
        for j in range(i + 1, n):
            dx = pos[j] - pos[i]
            f = mass[i] * mass[j] / (dx * dx + 1e-9)
            forces[i] = forces[i] + f
            forces[j] = forces[j] - f
        label = f"particle {i}"         # f-string
    return np.array(forces)
```

Listing 4. N-body kernel as written by the developer.

Submitting this function to `@njit` directly produces a `TypingError` on both the list comprehension and the f-string. With `@jit()`, the CloudSealed Compiler rewrites both constructs before Numba compilation, and the function compiles and executes correctly.

Table II reports the wall-clock time for 300 calls to `compute_forces` with $n = 100$ particles, comparing the compiled version against a pure-Python baseline on the same machine used for the AST overhead measurements.

TABLE II
N-BODY KERNEL PERFORMANCE: `@JIT()` VS. PURE PYTHON ($n = 100$ PARTICLES, 300 CALLS, X86-64 LINUX, PYTHON 3.14, NUMBA 0.68).

Implementation	Median (µs)	p99 (µs)
Pure Python	3,121	—
`@jit()` (compiled)	14	21
Speedup	**220**×	

The developer writes one version of the code, in idiomatic Python 3. The 220× speedup is achieved without any manual rewrite of the list comprehension or removal of the f-string label.

C. AST Transform Overhead

Table III reports the one-time overhead of each transform for a representative 15-line function containing one f-string and one list comprehension.

TABLE III
ONE-TIME AST TRANSFORM OVERHEAD MEASURED AT DECORATION TIME (1,000 REPETITIONS, X86-64 LINUX, PYTHON 3.11.9).

Transform	Median (µs)	p99 (µs)
FStringTransformer	681	1,650
ListCompTransformer	682	1,380
Both combined	1,380	2,900

This overhead is paid once per function at decoration time and is not incurred on subsequent calls to the compiled function.

D. Kwargs Dispatch Overhead

Table IV reports per-call overhead introduced by the kwargs unroller for a trivial compiled function (`return a + b`).

TABLE IV
PER-CALL OVERHEAD OF THE KWARGS UNROLLING WRAPPER.

Call style	Median (µs/call)
Positional args (baseline)	0.070
Keyword args	0.121
Wrapper overhead (delta)	0.051

The 51 ns overhead per call is negligible relative to any non-trivial compiled function body.

E. Diagnostic Pattern Coverage

The diagnostic layer checks for six categories of unsupported Numba patterns by scanning the function source string: set construction, `with` statements, `yield` expressions, dictionary unpacking (`{**d}`), inline class definitions, and `import` statements. A confirmed test case (set construction in nopython mode) produces the expected `CloudSealedCompileError` with the correct hint. The current implementation uses string matching rather than AST-structural analysis, which means patterns inside nested functions may be missed; this is a known limitation noted in Section V.

F. Test Suite

The 38-case test suite covers all six contributions: 5 cases for `FStringTransformer` (including format specs and multiple expressions); 5 cases for `ListCompTransformer` (including filter clauses and execution-correctness verification); 9 cases for the type mapper (scalars, arrays, `Optional`, `List`, `Tuple`, `Dict`, string fallbacks, unsupported types, and Numba-unavailable path); 7 cases for the `@jit` decorator (basic compilation, kwargs, array input, stats recording, fallback, untyped functions, name preservation); 1 diagnostic case; 5 `@jitdataclass` cases; and 6 compilation stats cases. All tests pass under Python 3.10–3.13 with and without Numba installed.

V. LIMITATIONS AND THREATS TO VALIDITY

Source extraction dependency. The approach depends on `inspect.getsource()`, which fails for functions defined in interactive sessions, created via `exec()`, or compiled from `.pyc` files without a corresponding `.py` source. In these cases the decorator falls back to calling `njit` on the original function unchanged, which may then fail with a raw Numba error if the function contains unsupported constructs.

Multi-generator comprehensions. The `ListCompTransformer` rewrites only single-generator comprehensions. Nested comprehensions (`[f(x,y) for x in a for y in b]`) are left untransformed.

Format-spec loss. F-string format specifiers (`.2f`, `>10`) are dropped silently. This is a correctness trade-off: the numeric

value is still included in the output string, but its formatting is lost.

Benchmark scope. AST transform benchmarks measure overhead for a single 15-line function. Functions with very large source bodies would incur proportionally higher parsing overhead. The kwargs overhead was measured with a minimal function body; the relative overhead would be smaller for functions with substantial computation.

Diagnostic coverage. The diagnostic layer uses source-string pattern matching, not structural AST analysis. Unsupported patterns inside nested functions or closures may not be detected. The test suite includes one confirmed diagnostic test case; a broader evaluation across the Numba issue tracker would provide stronger evidence of coverage.

VI. Conclusion

Numba is fast. The problem has never been its performance; it has been the tax it imposes on the code around it. The Cloud-Sealed Compiler eliminates that tax by intercepting functions before Numba ever sees them, rewriting the constructs nopython mode cannot handle, and mapping the type information that was already there in PEP 484 annotations into the signatures Numba needs.

The result is that developers write one version of their code, in the Python they already know, and the compiler handles the rest. When Numba is not installed, the package returns the original function unchanged and execution continues in pure Python. The one-time overhead at decoration time is below 1.4 ms; the per-call kwargs overhead is 51 ns.

Three concrete directions remain open. The `ListCompTransformer` currently handles single-generator comprehensions; nested generators require a more involved rewrite that preserves scoping semantics. The diagnostic layer uses source-string matching, which misses patterns inside closures; replacing it with AST structural analysis would improve coverage. Format specifiers in f-strings are silently dropped today because Numba cannot format floats; as Numba's string support matures, this trade-off can be revisited.

References

[1] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proc. 2nd Workshop LLVM Compiler Infrastructure in HPC*, 2015, pp. 1–6.

[2] T. Zhang, X. Liu, and Y. Luo, "Python meets JIT compilers: A simple implementation and a comparative evaluation," *Softw. Pract. Exp.*, vol. 54, no. 3, pp. 412–431, 2024.

[3] T. E. Oliphant, "Python for scientific computing," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 10–20, May 2007.

[4] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, "Cython: The best of both worlds," *Comput. Sci. Eng.*, vol. 13, no. 2, pp. 31–39, Mar. 2011.

[5] D. S. Seljebotn, "Fast numerical computations with Cython," in *Proc. Python in Science Conf. (SciPy)*, 2009, pp. 15–22.

[6] J. Lehtosalo *et al.*, "mypyc: Compiling Python to C using mypy type information," in *Proc. Python in Science Conf. (SciPy)*, 2022, pp. 44–50.

[7] A. Rigo and S. Pedroni, "PyPy's approach to virtual machine construction," in *Proc. Dynamic Languages Symp. (DLS)*, 2006, pp. 944–953.

[8] C. F. Bolz-Tereick and A. Rigo, "Tracing the meta-level: PyPy's tracing JIT compiler," in *Proc. ICOOOLPS*, 2009, pp. 18–25.

[9] I. Cosenza, N. Gowanlock, B. Juurlink, and P. Millet, "numba-dpex: A data-parallel extension for Numba," *SoftwareX*, vol. 18, p. 101040, 2022.

[10] Python Software Foundation, "`ast` — Abstract Syntax Trees," Python 3.11 Documentation, 2023. [Online]. Available: https://docs.python.org/3/library/ast.html

[11] G. van Rossum and P. J. Eby, "PEP 484 — Type Hints," Python Enhancement Proposals, 2014. [Online]. Available: https://peps.python.org/pep-0484/

[12] L. Rauchwerger and D. Padua, "The LRPD test: Speculative run-time parallelization of loops with privatization and reduction parallelization," *IEEE Trans. Parallel Distrib. Syst.*, vol. 10, no. 2, pp. 160–180, Feb. 1999.

[13] Y. Liu, S. Huang, and C. Ding, "A performance study of LLVM-based just-in-time compilation for dynamically typed languages," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 7, pp. 1873–1887, Jul. 2017.

Rodrigo Martinez Pinto is a software engineer focused on high-performance Python and cloud cost-optimization tooling. He has contributed bug fixes and features to the Numba upstream compiler (pull requests #10880, #10881, and #10882, merged into the `numba/numba` repository) and is the author of the open-source package `cloudsealed-jit`. Contact: contact@cloudsealed.com